

Análisis Técnico de la Feature Administrativa de Reservas

Principios SOLID, Patrones de Diseño y Arquitectura

Fecha del análisis: 2026-04-09

Ámbito: Backend (Java 17 + Spring Boot) y Frontend (React + TypeScript)

Enfoque: HU-01 (Consultar reservas desde módulo administrador) y HU-02 (Crear reserva manual desde módulo administrador)

Glosario de Siglas

- **HU:** Historia de Usuario.
- **SOLID:** Conjunto de principios de diseño orientados a mantenibilidad.
- **SRP (Single Responsibility Principle):** una clase/componente debe tener una sola razón de cambio.
- **OCP (Open/Closed Principle):** el diseño debe permitir extensión sin modificar comportamiento estable.
- **LSP (Liskov Substitution Principle):** una implementación debe poder sustituir a otra sin romper contratos.
- **ISP (Interface Segregation Principle):** interfaces pequeñas y enfocadas, no contratos inflados.
- **DIP (Dependency Inversion Principle):** la capa de aplicación depende de abstracciones, no de detalles técnicos.
- **CQRS (Command Query Responsibility Segregation):** separar operaciones de lectura y de escritura.
- **DTO (Data Transfer Object):** objeto de transporte entre capas/protocolos.
- **API (Application Programming Interface):** contrato de integración entre consumidores y servicios.
- **JWT (JSON Web Token):** token para identidad/autorización.
- **STOMP (Simple Text Oriented Messaging Protocol):** protocolo de mensajería sobre WebSocket.

Introducción Contextual

La feature analizada introduce un módulo administrativo con dos capacidades encadenadas:

1. **HU-01:** consultar reservas por listado paginado, filtros y detalle.
2. **HU-02:** crear reservas manuales para terceros, con validaciones funcionales y de disponibilidad.

Este documento conserva la estructura y profundidad del `resultado_tecnico.md`, pero acota el análisis a la porción de arquitectura realmente tocada por ambas HU. El propósito no es inventariar clases, sino explicar por qué las decisiones de diseño sí corresponden a principios SOLID y patrones concretos, con trazabilidad a archivos del proyecto.

I. BACKEND - ANÁLISIS POR SERVICIO

1. AUTH-SERVICE: Autenticación, autorización y catálogo de usuarios no admin

Descripción General

`auth-service` sostiene dos responsabilidades críticas de la feature: validar que el actor tenga perfil administrativo y exponer un catálogo de usuarios elegibles para reservas manuales (excluyendo rol admin). No crea reservas, pero define quién puede crearlas y para quién.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- **AuthController** concentra la traducción HTTP de acciones de autenticación y del endpoint administrativo de consulta de usuarios.
- **Archivo relacionado:** `Backend/services/auth-service/src/main/java/com/reservas/sk/auth_service/adapters/in/web/AuthController.java`
- **Justificación:** la clase no decide persistencia ni reglas de hash/token; su razón de cambio es el contrato web.

- **AuthService** orquesta casos de uso (login, register, getMe, listNonAdminUsers) sin detalles de JPA o filtros SQL.
- **Archivo relacionado:** Backend/services/auth-service/src/main/java/com/reservas/sk/auth_service/application/service/AuthApplicationService.java
- **Justificación:** su razón de cambio es la política de negocio de autenticación y catálogo de usuarios.
- **UserPersistenceAdapter** encapsula la consulta de usuarios no admin en infraestructura de persistencia.
- **Archivo relacionado:** Backend/services/auth-service/src/main/java/com/reservas/sk/auth_service/adapters/out/persistence/UserPersistenceAdapter.java
- **Justificación:** separa mapeo de entidad/rol del comportamiento de aplicación.

O - Open/Closed Principle (OCP)

- El caso de uso se extiende con listNonAdminUsers sin alterar contratos previos de login y registro.
- **Archivos relacionados:**
 - .../application/port/in/AuthUseCase.java
 - .../application/service/AuthApplicationService.java
- **Justificación:** la funcionalidad nueva entra como operación adicional, no como mutación destructiva de flujos existentes.

L - Liskov Substitution Principle (LSP)

- Implementaciones de publicación de eventos (RabbitUserEventPublisherAdapter y fallback NoOp) permanecen sustituibles para la capa de aplicación.
- **Archivos relacionados:**
 - .../application/port/out/UserEventPublisherPort.java
 - .../adapters/out/messaging/RabbitUserEventPublisherAdapter.java
 - .../adapters/out/messaging/NoOpUserEventPublisherAdapter.java
- **Justificación:** el servicio de aplicación no requiere condicionales por implementación.

I - Interface Segregation Principle (ISP)

- El puerto UserPersistencePort mantiene métodos focalizados (buscar/guardar/listar no admin) sin mezclar tokenización o mensajería.
- **Archivo relacionado:** Backend/services/auth-service/src/main/java/com/reservas/sk/auth_service/application/port/out/UserPersistencePort.java
- **Justificación:** consumidores dependen de capacidades mínimas para su caso de uso.

D - Dependency Inversion Principle (DIP)

- AuthService depende de UserPersistencePort, PasswordHasherPort, TokenPort, UserEventPublisherPort.
- **Archivo relacionado:** Backend/services/auth-service/src/main/java/com/reservas/sk/auth_service/application/service/AuthApplicationService.java
- **Justificación:** la aplicación no conoce SpringDataUserRepository ni librerías JWT/Bcrypt.

Patrones de Diseño Implementados

Hexagonal Architecture (Ports & Adapters)

- Entrada web en AuthController, orquestación en AuthService, salida por puertos de persistencia/seguridad/eventos.
- **Valor en HU-02:** habilita el endpoint de usuarios no admin sin acoplar UI admin a consultas directas de BD.

Adapter Pattern

- UserPersistenceAdapter traduce entidades (UserJpaEntity, RoleJpaEntity) a dominio User.
- **Valor en HU-02:** la pantalla administrativa recibe nombres/correos limpios en vez de estructura persistente interna.

Strategy Pattern

- Estrategias de hash/token/eventos quedan intercambiables vía puertos.
- **Valor funcional:** el módulo admin no queda bloqueado por un solo mecanismo de publicación o criptografía.

2. BOOKINGS-SERVICE: Núcleo de consulta administrativa y creación manual

Descripción General

bookings-service ejecuta el corazón de HU-01/HU-02: filtrado paginado por criterios administrativos, enriquecimiento de detalle, validación de solapamiento y alta manual con estado inicial esperado por operación.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- **BookingController** se centra en exponer endpoints y normalizar request/response de capa web.
- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/adapters/in/web/BookingController.java
- **Justificación:** no contiene SQL ni reglas de validación temporal compleja.
- **BookingApplicationService** concentra reglas de negocio: paginación segura, validación de filtros de fecha/estado, conflicto horario, validación de equipos.
- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/application/service/BookingApplicationService.java
- **Justificación:** es el punto de verdad de políticas funcionales HU-01/HU-02.
- **JdbcBookingPersistenceAdapter** encapsula consultas SQL y joins para enriquecer datos administrativos (usuario, sede, espacio).
- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/adapters/out/persistence/JdbcBookingPersistenceAdapter.java
- **Justificación:** separa acceso relacional del modelo de dominio.
- **ReservationStatusCatalog** concentra normalización y mapeo de estados funcionales.
- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/domain/model/ReservationStatusCatalog.java
- **Justificación:** evita dispersar el catálogo en controller/service/sql.

O - Open/Closed Principle (OCP)

- Se añade AdminListReservationsQuery y operaciones administrativas sin romper rutas de consulta de usuario.
- **Archivos relacionados:**
 - .../application/usecase/AdminListReservationsQuery.java
 - .../application/port/in/BookingUseCase.java
- **Justificación:** la extensión entra como variante de consulta, no como reescritura del flujo existente.

L - Liskov Substitution Principle (LSP)

- ReservationEventPublisherPort y ReservationRealtimePort mantienen implementación sustituible.
- **Archivos relacionados:**
 - .../application/port/out/ReservationEventPublisherPort.java
 - .../adapters/out/messaging/RabbitReservationEventPublisherAdapter.java
 - .../adapters/out/messaging/NoOpReservationEventPublisherAdapter.java
 - .../adapters/out/websocket/StompReservationRealtimeAdapter.java
- **Justificación:** la aplicación no necesita conocer el canal de publicación para funcionar.

I - Interface Segregation Principle (ISP)

- BookingPersistencePort discrimina operaciones de lectura admin (listAdminReservations, countAdminReservations) respecto a comandos mutables.

- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/application/port/out/BookingPersistencePort.java
- **Justificación:** evita interfaces monolíticas y aclara intención.

D - Dependency Inversion Principle (DIP)

- BookingApplicationService depende de puertos de persistencia y mensajería.
- **Archivo relacionado:** Backend/services/bookings-service/src/main/java/com/reservas/sk/bookings_service/application/service/BookingApplicationService.java
- **Justificación:** habilita cambios de infraestructura sin tocar reglas HU.

Patrones de Diseño Implementados

Hexagonal Architecture

- Controller -> UseCase -> ApplicationService -> Ports -> Adapters.
- **Aporte HU-01:** consulta admin paginada con filtros.
- **Aporte HU-02:** creación manual con validación previa y publicación de evento.

CQRS Parcial

- **Queries:** AdminListReservationsQuery, ListReservationsQuery, CheckSpaceAvailabilityQuery.
- **Commands:** CreateReservationCommand, UpdateReservationCommand, HandoverReservationCommand.
- **Justificación:** lectura y escritura evolucionan con restricciones diferentes.

Repository Pattern

- Puerto BookingPersistencePort con implementación JDBC.
- **Aporte HU-01:** joins de usuarios/espacios/ciudades para mostrar nombres de negocio.

Mapper Pattern

- BookingHttpMapper transforma dominio hacia DTO admin (AdminReservationResponse, AdminReservationsPageResponse).
- **Aporte HU-01:** consistencia entre tabla y detalle.

State/Value Catalog Pattern (implícito)

- ReservationStatusCatalog gobierna validación de estados admin y del sistema.
- **Aporte HU-01:** filtro de estado confiable y no ambiguo.

3. INVENTORY-SERVICE: Catálogo de equipos para la creación manual

Descripción General

La HU-02 utiliza inventario como soporte: no para decidir la reserva principal, sino para poblar equipos adicionales compatibles con la ciudad seleccionada.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- EquipmentsController expone operaciones HTTP de equipos.
- **Archivo relacionado:** Backend/services/inventory-service/src/main/java/com/reservas/sk/inventory_service/adapters/in/web/EquipmentsController.java
- InventoryApplicationService valida estado/cityId y reglas de catálogo.
- **Archivo relacionado:** Backend/services/inventory-service/src/main/java/com/reservas/sk/inventory_service/application/service/InventoryApplicationService.java

O - Open/Closed Principle (OCP)

- ListEquipmentsQuery permite evolución de filtros sin romper endpoint base.

- **Archivo relacionado:** Backend/services/inventory-service/src/main/java/com/reservas/sk/inventory_service/application/usecase/ListEquipmentsQuery.java

D - Dependency Inversion Principle (DIP)

- La aplicación depende de puertos de persistencia/eventos.
- **Archivo relacionado:** Backend/services/inventory-service/src/main/java/com/reservas/sk/inventory_service/application/service/InventoryApplicationService.java

Patrones de Diseño Implementados

Hexagonal + Adapter

- Controlador y servicio aislados del motor de persistencia.
- **Aporte HU-02:** el frontend admin consume catálogo sin acoplamiento a tablas internas.

Mapper Pattern

- InventoryHttpMapper traduce representación de transporte.
- **Archivo relacionado:** Backend/services/inventory-service/src/main/java/com/reservas/sk/inventory_service/adapters/in/web/InventoryHttpMapper.java

4. LOCATIONS-SERVICE: Cadena ciudad -> espacios

Descripción General

locations-service aporta el comportamiento dependiente necesario para HU-02: primero ciudad/sede, luego espacios/salas asociados.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- LocationsController separa rutas de ciudades y espacios, incluyendo /cities/{cityId}/spaces.
- **Archivo relacionado:** Backend/services/locations-service/src/main/java/com/reservas/sk/locations_service/adapters/in/web/LocationsController.java
- LocationsApplicationService concentra validaciones de existencia y consistencia de ciudad/espacio.
- **Archivo relacionado:** Backend/services/locations-service/src/main/java/com/reservas/sk/locations_service/application/service/LocationsApplicationService.java

O - Open/Closed Principle (OCP)

- ListSpacesQuery habilita filtro por ciudad y por activo sin reescribir flujos previos.
- **Archivo relacionado:** Backend/services/locations-service/src/main/java/com/reservas/sk/locations_service/application/usecase/ListSpacesQuery.java

I - Interface Segregation Principle (ISP)

- LocationsPersistencePort y LocationEventPublisherPort mantienen contratos claros y separados.
- **Archivos relacionados:**
 - .../application/port/out/LocationsPersistencePort.java
 - .../application/port/out/LocationEventPublisherPort.java

D - Dependency Inversion Principle (DIP)

- Servicio de aplicación depende de puertos, no de acceso SQL directo.

Patrones de Diseño Implementados

Hexagonal Architecture

- Repite patrón de entrada/salida desacoplada.

Query Object Pattern

- ListSpacesQuery encapsula criterios de consulta para mantener controller liviano.

5. API-GATEWAY: Borde de integración

Descripción General

La feature admin no expone puertos internos de microservicios al frontend; se enruta desde una única entrada.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- GatewayProxyConfig define reglas de enrutamiento y nada más.
- **Archivo relacionado:** Backend/services/api-gateway/src/main/java/com/reservas/sk/api_gateway/infrastructure/config/GatewayProxyConfig.java

D - Dependency Inversion Principle (DIP)

- URLs de servicios viven en propiedades (GatewayRoutesProperties, application.properties), no incrustadas en lógica.
- **Archivos relacionados:**
 - .../infrastructure/config/GatewayRoutesProperties.java
 - Backend/services/api-gateway/src/main/resources/application.properties

Patrones de Diseño Implementados

Gateway/Facade Pattern

- El frontend consume /auth/**, /bookings/**, /locations/**, /inventory/** por un único punto.

Strangler Pattern (operativo)

- Se extiende funcionalidad admin sin fracturar rutas legacy.

6. NOTIFICATIONS-SERVICE: Canal de difusión en tiempo real

Descripción General

Aunque HU-01/HU-02 no depende exclusivamente de WebSocket para cerrar el flujo, el servicio de notificaciones sostiene el modelo reactivo de eventos administrativos.

Principios SOLID Identificados

S - Single Responsibility Principle (SRP)

- RabbitEventListenerAdapter: escucha eventos AMQP.
- StompWebSocketBroadcastAdapter: publica al canal WebSocket.
- **Archivos relacionados:**
 - Backend/services/notifications-service/src/main/java/com/reservas/sk/notifications_service/adapters/in/rabbit/RabbitEventListenerAdapter
 - Backend/services/notifications-service/src/main/java/com/reservas/sk/notifications_service/adapters/out/websocket/StompWebSocketBroadcast

D - Dependency Inversion Principle (DIP)

- EventBroadcastApplicationService trabaja con WebSocketBroadcastPort.

- **Archivo relacionado:** Backend/services/notifications-service/src/main/java/com/reservas/sk/notifications_service/application/service/EventBroadcastApplicationServi

Patrones de Diseño Implementados

Observer / Pub-Sub

- Recepción en broker y retransmisión a suscriptores WebSocket.

Adapter Pattern

- Adaptadores de entrada/salida desacoplan protocolos de transporte.

II. FRONTEND - ANÁLISIS ARQUITECTÓNICO

Descripción General

El frontend incorpora una vista administrativa específica (/admin-reservations) con control de acceso por rol, filtros operativos, paginación, detalle, creación manual y notificación visual.

Estructura Arquitectónica

```
routes/  
└─ AppRouter.jsx                # Ruteo + AdminRoute  
  
ui/components/common/  
└─ AdminRoute.jsx              # Guardia por rol  
  
ui/pages/admin-reservations/  
└─ AdminReservationsPage.jsx    # Orquestación de HU-01 y HU-02  
  
ui/components/admin-reservations/  
├─ AdminCreateReservationPanel.jsx  
├─ ReservationRequesterAutocomplete.jsx  
└─ ReservationStatusFilter.jsx  
  
core/adapters/hooks/  
├─ useAdminReservationFilters.ts  
└─ useAdminCreateReservationForm.ts  
  
features/reservations/services/  
└─ adminReservationsService.js
```

Principios SOLID en Frontend

S - Single Responsibility Principle

- AdminRoute.jsx solo resuelve autorización por token/rol.
- useAdminReservationFilters.ts solo maneja estado y validación de filtros.
- useAdminCreateReservationForm.ts solo maneja estado/errores del formulario.
- adminReservationsService.js solo concentra integración HTTP.

O - Open/Closed Principle

- La pantalla AdminReservationsPage.jsx se extiende por hooks y servicios; agregar un nuevo filtro/campo no exige romper toda la vista.

L - Liskov Substitution Principle

- El modal reutilizado opera en modo creación y modo detalle respetando contrato de props, sin romper comportamiento del contenedor.

I - Interface Segregation Principle

- Hooks admin exponen API mínima:
- filtros: `setFilter`, `applyFilters`, `clearFilters`
- formulario: `setField`, `validateForm`, `canSubmit`, `resetForm`

D - Dependency Inversion Principle

- La página depende de funciones de servicio (`getAdminReservations`, `createAdminReservation`, etc.), no de axios o `fetch` embebido en componentes.

Patrones de Diseño en Frontend

Service Layer Pattern

- `adminReservationsService.js` abstrae endpoints, parsing y fallbacks.

Facade Pattern (UI)

- `AdminReservationsPage.jsx` orquesta tabla, filtros, modal y feedback como punto de entrada único de la feature.

Adapter/Mapper Pattern

- El servicio transforma respuestas del backend a estructura consumible por tabla y detalle.

Observer Pattern (ligero)

- Toasts y recargas de tabla reaccionan al resultado de operaciones asíncronas.

Integración Frontend-Backend

- Consulta HU-01:
- `GET /bookings/admin/reservations?page=0&size=20&...filtros`
- `GET /bookings/reservations/{id}`
- Catálogos HU-02:
- `GET /auth/users?query=...` (no admin)
- `GET /locations/cities`
- `GET /locations/cities/{cityId}/spaces`
- `GET /inventory/equipments?cityId=...`
- Creación HU-02:
- `POST /bookings/reservations`
- validación de solapamiento y manejo de mensajes de error/éxito.

III. PATRONES TRANSVERSALES

1. Command Query Responsibility Segregation (CQRS) - Parcial

Backend - Lectura admin: `AdminListReservationsQuery`. - Escritura admin: `CreateReservationCommand`.

Frontend - Lectura: carga de tabla y detalle. - Escritura: `submit` del modal administrativo.

2. Event-Driven Architecture

La creación/modificación de reservas puede emitir eventos que viajan por mensajería y se reflejan en interfaces suscritas. La feature no depende de un polling rígido para mantener coherencia operativa.

3. Strangler Pattern (API Gateway)

La entrada única del gateway permite crecer el módulo admin sin dispersar cambios de URL o puertos en el cliente.

4. Circuit Breaker Pattern (Implícito)

No está materializado como componente explícito en las HU, pero la separación por puertos permite introducir resiliencia por adaptador sin invadir dominio.

5. Mapper Pattern (Bidireccional)

- Backend: mapeo dominio -> DTO HTTP admin.
- Frontend: mapeo payload API -> view model.

IV. VERIFICACIÓN DE ESTADOS DE RESERVA

Estados Identificados en Backend

ReservationStatusCatalog normaliza estados para operación administrativa.
Estados funcionales observables en HU:

- Pendiente
- Confirmada
- Cancelada
- Finalizada

Además, el servicio valida el paso de estado según reglas de conflicto y de ventana temporal.

Estados en Frontend

La vista admin consume estados normalizados para:

- filtro por estado en HU-01;
- visualización de tabla y detalle;
- refresco post creación en HU-02 para confirmar estado operativo del registro recién creado.

V. CONCLUSIONES Y RECOMENDACIONES

Fortalezas Arquitectónicas

1. La feature se integró sin romper contratos existentes, conservando separación por capas.
2. HU-01/HU-02 quedaron trazables por servicio, con responsabilidades técnicas claramente acotadas.
3. El backend dejó de exponer información cruda para admin y prioriza datos de negocio legibles.
4. El frontend administra validaciones y feedback sin acoplar componentes a detalles de infraestructura.

Áreas de Mejora

1. Formalizar contrato único de catálogo de estados entre backend y frontend.
2. Incorporar métricas de uso del módulo admin (errores por validación, conflictos de horario, latencia por filtros).
3. Reutilizar patrón HU-01/HU-02 para las siguientes HU administrativas (reprogramación, cancelación masiva, auditoría de cambios).

VI. MATRIZ DE REFERENCIA: PRINCIPIOS SOLID Y PATRONES

Principio/Patrón	Auth-Service	Bookings-Service	Inventory-Service	Locations-Service	Api-Gateway	Notifications-Service	Frontend Admin
S - Single Responsibility	✓	✓	✓	✓	✓	✓	✓
O - Open/Closed	✓	✓	✓	✓	-	-	✓

Principio/Patrón	Auth-Service	Bookings-Service	Inventory-Service	Locations-Service	Api-Gateway	Notifications-Service	Frontend Admin
L - Liskov Substitution	✓	✓	-	-	-	-	✓
I - Interface Segregation	✓	✓	-	✓	-	-	✓
D - Dependency Inversion	✓	✓	✓	✓	✓	✓	✓
Hexagonal Architecture	✓	✓	✓	✓	Parcial	✓	Adaptada
Adapter Pattern	✓	✓	✓	✓	✓	✓	✓
Strategy Pattern	✓	✓	Parcial	Parcial	-	-	Parcial
Repository Pattern	✓	✓	✓	✓	-	-	-
CQRS (Parcial)	-	✓	-	-	-	-	✓
Event-Driven	✓	✓	✓	✓	-	✓	✓
Mapper Pattern	✓	✓	✓	✓	-	-	✓
Dependency Injection	✓	✓	✓	✓	✓	✓	✓

Fin del análisis técnico de la feature administrativa*Documento generado: 2026-04-09**Cobertura: HU-01 y HU-02 del módulo administrador*